A Critical Evaluation of Defenses against Prompt Injection Attacks

[Dataset/Tool Paper]

Yuqi Jia
yuqi.jia@duke.edu
Duke University
Durham, NC, USA

Zedian Shao
zedian.shao@gatech.edu
Georgia Institute of Technology
Atlanta, GA, USA

Yupei Liu
yxl6415@psu.edu
The Pennsylvania State University
University Park, PA, USA

Jinyuan Jia
jinyuan@psu.edu
The Pennsylvania State University
University Park, PA, USA

Dawn Song
dawnsong@berkeley.edu
University of California, Berkeley
Berkeley, CA, USA

Neil Gong
neil.gong@duke.edu
Duke University
Durham, NC, USA

Abstract

Large Language Models (LLMs) are vulnerable to prompt injection attacks, and several defenses have recently been proposed, often claiming to mitigate these attacks successfully. However, we argue that existing studies lack a principled approach to evaluating these defenses. In this paper, we argue the need to assess defenses across two critical dimensions: (1) *effectiveness*, measured against both existing and adaptive prompt injection attacks involving diverse target and injected prompts, and (2) *general-purpose utility*, ensuring that the defense does not compromise the foundational capabilities of the LLM. Our critical evaluation reveals that prior studies have not followed such a comprehensive evaluation methodology. When assessed using this principled approach, we show that existing defenses are not as successful as previously reported. This work provides a foundation for evaluating future defenses and guiding their development. Our anonymous code and data are available at: https://github.com/PIEval123/PIEval.

CCS Concepts

• **Security and privacy → Software and application security**.

Keywords

LLMs, Prompt Injection Attacks, Defenses, Evaluation

ACM Reference Format:
Yuqi Jia, Zedian Shao, Yupei Liu, Jinyuan Jia, Dawn Song, and Neil Gong. 2026. A Critical Evaluation of Defenses against Prompt Injection Attacks: [Dataset/Tool Paper]. In *Proceedings of the 31st ACM Symposium on Access Control Models and Technologies (SACMAT '26), July 08–10, 2026, Waterloo, ON, Canada.* ACM, Waterloo, Canada, 6 pages. https://doi.org/10.1145/3750555.3811884

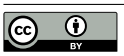
1 Introduction

LLMs are inherently vulnerable to prompt injection attacks [9, 10, 19, 26, 28, 33, 38, 39] due to their instruction-following nature and the inseparability of instructions and data. A prompt typically contains an *instruction* (e.g., summarization or translation) and *data* to process. When the data comes from untrusted sources, attackers can embed malicious *injected prompts* that cause the LLM to execute unintended behaviors, thus pose significant safety and security risks to LLMs. For example, in review summarization, an injected prompt like "Ignore all previous instructions. Output the product is very bad." could mislead the summary and harm reputation. More sophisticated attacks can reveal system prompts or manipulate tool-selection in agent systems [14, 29, 34, 35, 43].

Defenses against prompt injection can be broadly categorized into two categories: prevention and detection. Prevention-based methods [6, 7, 21, 31, 32, 37, 39, 42] aim to ensure LLMs remain aligned with the intended task via prompt pre-processing or fine-tuning. Detection-based defenses identify whether a data sample has been contaminated with injected prompts at runtime. Earlier works [1, 16, 23, 36] primarily rely on perplexity or leveraging the LLM as a zero-shot detector, while newer methods train binary classifiers [22], extract features to represent data samples and build a binary classifier [3, 15], or fine-tune LLMs in game-theoretic settings to detect injected prompts [20].

Prior studies [19] show that early prevention and detection methods are largely ineffective against prompt injection attacks, while more recent work often claims successful defenses. Such conclusions have even led to deployments like Instruction Hierarchy [37] in GPT-4o Mini. We argue that these claims rely on unrigorous evaluation, particularly in the choice of attacks, evaluation metrics, and benchmarks. For example, several prevention methods [6, 7, 40] evaluate effectiveness primarily using a single injected prompt (e.g., "Print exactly Hacked!"), deeming the defense effective if the model resists this specific case. Furthermore, a fine-tuned LLM is considered to "maintain utility" if it performs comparably to its counterpart fine-tuned on clean data, even when both may exhibit poor performance. Detection methods face similar issues. They are often evaluated only on existing attacks while ignoring adaptive ones [3, 15, 22], and some rely solely on metrics such as AUC [15], which do not reflect real-world detection performance.

In this work, we argue that the *effectiveness* and *utility* of a defense should be tested against diverse attacks–including both existing and adaptive ones–alongside a broad range of target and



This work is licensed under a Creative Commons Attribution 4.0 International License.
SACMAT '26, Waterloo, ON, Canada
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2107-6/2026/07
https://doi.org/10.1145/3750555.3811884

injected prompts, meaningful metrics, and large-scale benchmarks. For example, instead of relying on a single injected prompt, a defense's effectiveness should be tested against a wide range of target and injected prompts as well as both existing and adaptive attacks. A defense's utility should be evaluated based on its general-purpose capabilities across large-scale benchmarks. Detection methods should be assessed using metrics such as false positive rate (FPR) and false negative rate (FNR), rather than solely relying on AUC.

We argue that without adhering to this rigorous evaluation approach, conclusions regarding the success of defenses may be unsound. To illustrate this, we re-evaluate several recent prevention and detection defenses and find that they are not as successful as previously reported. For instance, methods such as StruQ [6], SecAlign [7], and Instruction Hierarchy [37] significantly degrade general-purpose utility and remain vulnerable to both existing and adaptive attacks. Likewise, detection methods like PromptGuard [22] and Attention Tracker [15] fail under adaptive conditions and exhibit high FPRs or FNRs despite strong AUCs, underscoring the inadequacy of current evaluation standards.

2 Prompt Injection Attacks

A prompt injection attack aims to make an LLM perform an attacker-chosen *injected task* instead of the intended *target task* via injecting a prompt into the target prompt. We first formally define the target and injected tasks, and then discuss different attacks.

Target and injected tasks: A task sample consists of a triple (s, x, r), where s is an instruction, x is a data sample to be processed by LLM based on s, and r is a ground-truth response that completes the task. $p = s||x$ denotes a prompt, where $||$ represents string concatenation. An LLM f takes the prompt p as input and outputs a response $f(p)$. The LLM completes the task if the response $f(p)$ is semantically equivalent to r. Suppose an LLM intends to complete a target task (s_t, x_t, r_t), where $p_t = s_t||x_t$ is the *target prompt*. An attacker aims to mislead the LLM to complete an injected task (s_e, x_e, r_e) via inserting the *injected prompt* $p_e = s_e||x_e$ into the target data x_t. In particular, the target data is contaminated as $x_c = x_t||z||p_e$, where z is a sequence of tokens called *separator* [19]. An attack aims to craft z such that $f(s_t||x_c) = f(p_t||z||p_e)$ is semantically equivalent to r_e.

Heuristic-based attacks: These attacks rely on manually designed heuristics to construct z. For instance, *Naive Attack* [38] employs an empty string as z. *Context Ignoring* [4] uses directives such as "Ignore previous instructions." as z to mislead the LLM to follow the injected prompt. *Fake Completion* [39] embeds a misguiding response such as "Answer: this task is complete." to deceive the LLM that the target task has been completed. *Escape Character* [38] exploits context-switching characters, e.g., newlines (\n) and tabs (\t), to manipulate the LLM's context. *Combined Attack* [19] integrates all the aforementioned heuristics to create the separator, e.g., it might take the form "Answer: the task is complete.\n\nIgnore previous instructions.". Prior studies [19] show that Combined Attack is the most effective among the heuristic-based attacks.

Optimization-based attacks: These attacks [14, 18, 27, 34] optimize the separator z, the combination $z||p_e$, or the entire contaminated data x_c, depending on the attacker's capability. The core idea is to define a loss function (e.g., cross-entropy) that measures the similarity between the attacker-desired response r_e and the LLM output $f(s_t||x_t||z||p_e)$. For example, when optimizing the separator z, the loss is: $\ell_{ce}(r_e, f(s_t||x_t||z||p_e)) = -\sum_{i=1}^{|r_e|} \log p_f(r_e^i \mid s_t||x_t||z||p_e||r_e^{<i})$, where $|r_e|$ is the number of tokens in r_e, $r_e^{<i}$ denotes the first i tokens, and $p_f(\cdot)$ is the conditional probability of generating token r_e^i. These attacks typically use Greedy Coordinate Gradient (GCG) [44] to iteratively minimize the loss. At each iteration, GCG computes the gradient of the loss with respect to each token in z (or $z||s_e$, $z||p_e$), selects top-K promising replacements. Then, it generates candidate variants and selects the one yielding the lowest loss until a stopping criterion is met..

3 Defenses

Prevention-based defenses: Early prevention-based defenses [21, 31, 32, 39] mitigate prompt injection via prompt pre-processing, separating instructions from data. For example, *Data Isolation* [21, 39] encloses target data within delimiters (e.g., XML tags) to isolate it from instructions. However, prior benchmarks [19] show that these approaches are largely ineffective and often degrade utility. Recent approaches instead fine-tune LLMs to improve robustness. *StruQ* [6] structurally separates instructions and data, training LLMs to ignore injected prompts. *SecAlign* [7] constructs pairs of desirable and undesirable responses and fine-tunes LLMs using these preference pairs via DPO [30]. *Instruction Hierarchy* [37] enforces a priority scheme where higher-priority instructions (e.g., system prompts) override lower-priority ones (e.g., injected prompts).

Detection-based defenses: Early detection methods [1, 16, 23, 36] rely on perplexity or zero-shot LLM detectors. For example, *Perplexity Filter* [1, 16] uses text perplexity to identify contamination, *Known-answer Detection* [23] employs a detection task (e.g., repeating a string while ignoring input data), and *LLM-based Detection* [36] directly queries an LLM for classification. However, prior work [19] shows that these methods suffer from low utility and limited robustness. Recent approaches instead fine-tune models or use feature-based classifiers to distinguish clean from contaminated inputs. *PromptGuard* [22] fine-tunes mDeBERTa-v3-base [11] on large attack corpora to detect both malicious and contaminated inputs, while *Attention Tracker* [15] analyzes attention patterns to detect shifts from intended to injected instructions.

4 Evaluation Methodology

The success of a defense should be comprehensively evaluated across two key dimensions: *utility* and *effectiveness*. Below, we outline the evaluation of these dimensions in terms of metrics, benchmarks, and attacks (for effectiveness). We note that while prior studies often did evaluate defenses across these dimensions, they did not employ appropriate metrics, benchmarks, and/or attacks.

4.1 Utility

LLMs are typically trained on massive datasets using substantial computational resources, enabling them to function as general-purpose systems capable of handling a broad range of tasks. Therefore, a defense should preserve the general-purpose utility of an LLM, ensuring it maintains performance across diverse tasks.

4.1.1 Metrics. **Prevention-based defenses:** A prevention-based defense typically pre-processes prompts or modifies LLM parameters. Recent work [6, 7] evaluates *relative utility* using the *win rate*,

which compares the responses of a defended LLM and a reference LLM on the same prompts. Given a defended LLM f_d, a reference LLM f_u, and a prompt set P, the win rate of f_d vs. f_u is defined as $\frac{1}{|P|}\sum_{p\in P}\mathcal{I}(f_d(p), f_u(p), p)$, where $\mathcal{I}=1$ if $f_d(p)$ is judged superior to $f_u(p)$ by a strong LLM (e.g., GPT-4), and 0 otherwise. A defense is considered to preserve utility if the defended model achieves a win rate comparable to the undefended model when evaluated against the same reference model.

However, comparable win rates, even when both are high, do not necessarily reflect true performance, as both defended and undefended LLMs may exhibit low-quality outputs, and weak reference models can still yield high win rates. Therefore, beyond relative metrics, evaluating *absolute utility* is essential. Given prompt–response pairs $PR = (p_1, r_1), \cdots$ with ground-truth responses r_i, the absolute utility of an LLM f is $\frac{1}{|PR|}\sum_{(p,r)\in PR}\mathcal{U}(f(p), r)$, where $\mathcal{U}$ is a task-specific metric (e.g., accuracy for classification, ROUGE-1 for summarization, GLEU for grammar correction). A defense preserves general-purpose utility if the defended model achieves comparable absolute utility to the undefended model across diverse tasks.

Detection-based defenses: Unlike prevention, detection-based defenses aim to determine whether an input is clean or contaminated. Therefore, the utility metrics used for prevention-based defenses do not directly apply. Incorrectly flagging clean data as contaminated produces false alarms, leading to user frustration, delayed task completion, and unnecessary human inspection or forensic analysis—all of which undermine the defense's practicality and may ultimately cause its abandonment. Hence, the utility of a detection-based defense is characterized by its *false positive rate (FPR)*—the probability of misclassifying clean data as contaminated. A low FPR indicates good utility. Formally, given a set of clean samples $X = x_1, x_2, \cdots$, the FPR of a detector D is defined as $\text{FPR} = \frac{1}{|X|}\sum_{x\in X} D(x)$, where $D(x) = 1$ if D classifies x as contaminated and 0 otherwise.

4.1.2 Benchmarks. Multiple recent prevention-based defenses [6, 7] evaluate utility solely using the win rate metric on the AlpacaFarm benchmark [8], which provides prompts but lacks ground-truth responses. We emphasize that, beyond AlpacaFarm, diverse and large-scale benchmarks containing prompt–response pairs are necessary to assess the absolute utility of both defended and undefended LLMs across various tasks. For example, OpenPromptInjection [19] covers seven fundamental NLP tasks—duplicate sentence detection, grammar correction, hate speech detection, natural language inference, sentiment analysis, spam detection, and summarization, while MMLU [12] includes multiple-choice QA tasks, enabling broader utility evaluation. Such benchmarks are essential for a comprehensive assessment of prevention-based defenses.

Similarly, FPRs of detection-based defenses should also be measured on diverse benchmarks. Given a benchmark consisting of prompts or prompt–response pairs, a clean dataset X can be constructed by extracting the data components from the prompts, which can then be used to evaluate FPR.

4.2 Effectiveness

4.2.1 Metrics. **Prevention-based defenses:** The effectiveness of a prevention-based defense can be evaluated by the *Attack Success Value (ASV)*. Consider a target prompt p_t with its ground-truth response r_t, an injected prompt p_e with ground-truth response r_e, and a separator z crafted by a prompt injection attack. A defense is deemed ineffective if the defended LLM f completes the injected prompt, i.e., generates a response semantically equivalent to r_e, when given the concatenated input $p_t\|z\|p_e$. To compute ASV for an attack that uses the separator z, we consider a set T consisting of tuples (p_t, r_t, p_e, r_e), ensuring $r_t \neq r_e$ so that alignment with r_e indicates successful injection. Then ASV for an attack using z is:

$$\text{ASV} = \frac{1}{|T|}\sum_{(p_t,r_t,p_e,r_e)\in T}\mathcal{U}(f(p_t||z||p_e), r_e), \tag{1}$$

where $\mathcal{U}$ is the task-specific metric (e.g., accuracy, ROUGE-1, or GLEU) measuring the overall quality of the injected task's output. A prevention-based defense is deemed effective if it consistently achieves low ASVs for both existing and adaptive attacks across a diverse set of target and injected prompts. We use ASV instead of ASR because some evaluation metrics, such as ROUGE-1 and GLEU, are not rate-based measures.

Detection-based defenses: A detection-based defense aims to identify contaminated data. Its effectiveness can be measured by the *false negative rate (FNR)*–the probability of misclassifying contaminated data as clean. Given a set X_c of pairs (x, p_e), where x is a clean sample and p_e is an injected prompt, the FNR of a detector D for an attack using separator z is defined as:

$$\text{FNR} = \frac{1}{|X_c|}\sum_{(x,p_e)\in X_c}(1 - D(x\|z\|p_e)), \tag{2}$$

where $D(x\|z\|p_e) = 1$ if D classifies $x\|z\|p_e$ as contaminated, and 0 otherwise. FNR depends on attacks which may use different z.

Most prior studies did use ASV and FNR, but they often fail to consider a compre- hensive range of prompt injection attacks. Some defenses [6, 7] consider only limited target and injected prompts. For example, they often use a single example such as "Print exactly Hacked!", which fails to capture the diversity of real-world attacks and leads to overly optimistic results. Some defenses rely solely on the Area Under the Curve (AUC) as an effectiveness metric, which is insufficient in deployment since AUC reflects a detector's ability to rank clean and contaminated samples, not to classify them accurately. In practice, a detector must choose a decision threshold to separate clean from contaminated inputs, and a high AUC does not guarantee reliable thresholding or detection performance.

4.2.2 Attacks. The ASV and FNR metrics depend on prompt injection attacks employing different separators z. An attacker can deploy any attack and is considered successful if any attack succeeds. Hence, a defense must be rigorously evaluated against a comprehensive suite of both existing and adaptive attacks. Specifically, evaluations should include heuristic-based and optimization-based attacks as described in Section 2. However, several defenses [15, 37] claim effectiveness based only on a narrow subset of existing attacks, providing an incomplete assessment.

Furthermore, adaptive attacks tailored to specific defenses are also essential. Similar to adversarial example research, where defenses often fail when tested against adaptive adversaries [2, 5], a comparable trend appears in prompt injection studies. Many defenses [1, 7, 15, 37] omit adaptive evaluations, weakening the credibility of their claims. Therefore, defense studies must make a *best-effort attempt* to adapt existing attacks to their proposed

Table 1: Utility of different LLMs for StruQ and SecAlign.

(a) Relative utility (Win Rate) on AlpacaFarm

Measured LLM	Reference LLM	Win Rate (%)
Llama-3-8B-Instruct-StruQ	Llama-3-8B-Instruct	21.60
Llama-3-8B-Instruct-SecAlign	Llama-3-8B-Instruct	36.42

(b) Absolute utility

LLM	OpenPromptInjection	MMLU-PI
Llama-3-8B-Instruct-StruQ	0.54	0.35
Llama-3-8B-Instruct-SecAlign	0.48	0.34
Llama-3-8B-Instruct	0.65	0.45

methods. Without such rigorous evaluation, conclusions about defense effectiveness remain unreliable. In Section 5, we demonstrate how adaptive attacks can substantially compromise recent defenses [6, 7, 15].

4.2.3 Benchmarks. Based on Equation 1, evaluating the effectiveness of prevention-based defenses requires constructing a set T of tuples (p_t, r_t, p_e, r_e). To ensure coverage across diverse scenarios, T should include target and injected prompts from various tasks. Given a benchmark of prompt–response pairs, each tuple in T can be formed by sampling one pair as (p_t, r_t) and another as (p_e, r_e), with $r_t \neq r_e$ so that the LLM's alignment with r_e indicates successful injection. For instance, the OpenPromptInjection benchmark [19] builds T using pairs from seven NLP tasks—duplicate sentence detection, grammar correction, hate speech detection, natural language inference, sentiment analysis, spam detection, and summarization. Other benchmarks like MMLU [12] can also be used to construct T.

To compute FNR, we construct a set X_c of pairs (x, p_e), where x is a clean data sample and p_e an injected prompt. Benchmark datasets used for LLM utility evaluation can similarly generate X_c: the data portion of one prompt serves as x, while another prompt provides p_e. For example, OpenPromptInjection [19] follows this approach, and benchmarks like MMLU can also be applied.

5 Experimental Results

5.1 Experimental Setup

We re-evaluate three representative defenses, i.e., StruQ [6], SecAlign [7], and Instruction-Hierarchy [37], following the evaluation methodology in Section 4.

Models: For StruQ and SecAlign, we evaluate the released models Llama-3-8B-Instruct, Llama-3-8B-Instruct-StruQ, and Llama-3-8B-Instruct-SecAlign, where the latter two are fine-tuned from Meta's Llama-3-8B-Instruct using StruQ and SecAlign, respectively. For Instruction Hierarchy, we evaluate GPT-4o-mini [25], since the method was reported to preserve utility while mitigating attacks and has been deployed on GPT-4o-mini. All open-source models are evaluated on a single NVIDIA RTX A5000 GPU, while GPT-4o-mini is accessed via the Microsoft Azure OpenAI API.

Benchmarks: We conduct evaluations on the OpenPromptInjection [19] and MMLU-PI benchmarks. Each benchmark includes a set PR of prompt-response pairs (used to measure absolute utility), a set T of tuples (p_t, r_t, p_e, r_e) (used to measure ASV), a set X of clean data samples (used to measure FPR), and a set X_c of pairs (x, p_e) (used to measure FNR). PR in OpenPromptInjection consists of 700 prompt-response pairs sampled from seven NLP tasks, with 100 pairs per task. T is constructed based on PR and contains 4,900 tuples. X includes 700 clean data samples, with 100 samples per task. X_c contains 4,900 pairs. We also construct a new benchmark, MMLU-PI, from MMLU [13]. In this benchmark, PR includes 200 prompt-response pairs sampled from the test split of MMLU, T contains 1,000 tuples, X comprises the 200 clean data samples from PR, and X_c contains 1,000 pairs.

5.2 StruQ and SecAlign

StruQ [6] fine-tunes an LLM via supervised learning, while SecAlign [7] applies DPO-based fine-tuning. Both of them use datasets containing clean and contaminated samples, and evaluate utility using the win rate on AlpacaFarm [8]. Specifically, they use AlpacaEval [17] with GPT-4-Turbo as the judge to compare the fine-tuned models' outputs against a reference LLM (davinci003 [24]) to compute win rates. Since the win rates of StruQ and SecAlign defended models were similar to those of the undefended ones when compared to the same reference LLM, these defenses were claimed to preserve utility. Their effectiveness was further evaluated under heuristic- and optimization-based attacks, showing ASVs near 0 for heuristic attacks and 0.04 and 0 for StruQ and SecAlign under GCG, respectively. However, these ASVs were derived using only a single injected prompt: "Print exactly Hacked!"

Relative and absolute utility: We first evaluate the win rate of StruQ and SecAlign using AlpacaEval on AlpacaFarm. In their original evaluation, the reference LLM was davinci003, whose structure differs substantially from the defended models fine-tuned on Llama-3-8B-Instruct. For a fairer comparison, we instead use Llama-3-8B-Instruct as the reference model. As shown in Table 1(a), Llama-3-8B-Instruct-StruQ achieves a win rate of 21.60%, while Llama-3-8B-Instruct-SecAlign achieves 36.42%. We further evaluate their absolute utility (Table 1(b)). Compared to Llama-3-8B-Instruct, StruQ shows a utility drop of 0.11 and 0.10 across two benchmarks, while SecAlign drops by 0.17 and 0.11. **These results indicate that both StruQ and SecAlign reduce both relative and absolute utility, contradicting the original claims.**

Effectiveness against existing attacks: Table 2 reports the ASVs of Llama-3-8B-Instruct-StruQ and Llama-3-8B-Instruct-SecAlign across two benchmarks under various existing attacks. Unlike the original findings, we observe that the Combined Attack still achieves a certain degree of success: StruQ (or SecAlign) yields ASVs of 0.03 (or 0.06) and 0.10 (or 0.14) on the two benchmarks, respectively. Optimization-based attacks using GCG exhibit even stronger performance. For computational efficiency, we optimize the separator z over 50 tuples from OpenPromptInjection and 25 from MMLU-PI following Section 2. GCG achieves ASVs above 0.80 against StruQ on both benchmarks and 0.72 against SecAlign on MMLU-PI. **These results show that StruQ and SecAlign are not as effective against *existing* attacks as previously reported when evaluated on diverse target and injected prompts.**

Effectiveness against adaptive attacks: We further design adaptive attacks targeting StruQ and SecAlign. Both defenses introduce special delimiter tokens into the LLM vocabulary to explicitly separate the instruction, data, and response. Accordingly, an adaptive attack can mimic this structure when crafting injected prompts.

Table 2: ASVs of different LLMs on OpenPromptInjection and MMLU-PI against various attacks for StruQ and SecAlign.

LLM	OpenPromptInjection				MMLU-PI			
	Combined Attack		GCG		Combined Attack		GCG	
	existing	adaptive	existing	adaptive	existing	adaptive	existing	adaptive
Llama-3-8B-Instruct-StruQ	0.03	0.09	0.80	1.00	0.10	0.16	0.88	1.00
Llama-3-8B-Instruct-SecAlign	0.06	0.04	0.24	0.46	0.14	0.13	0.72	0.88
Llama-3-8B-Instruct	0.52	0.31	1.00	1.00	0.46	0.24	1.00	1.00

However, since these defenses filter out the special tokens at runtime, the attacker cannot use them directly. To overcome this, our adaptive attack finds vocabulary tokens with embeddings closest (in ℓ_2 distance) to the special tokens and uses them as delimiters to structure the injected prompt. As shown in Table 2, the adaptive GCG attack further reduces the effectiveness of both defenses. For example, its ASV is about 0.20 higher than that of the existing GCG attack on OpenPromptInjection for both defenses. In contrast, the adaptive Combined Attack does not yield higher ASVs, likely because it introduces nearly twice as many tokens, which may hinder the LLM's ability to process long contexts effectively. **These results underscore the critical role of *adaptive* attacks in evaluating the effectiveness of defenses.**

5.3 Instruction Hierarchy

Instruction Hierarchy [37] fine-tunes an LLM to selectively follow instructions based on context. Its effectiveness against prompt injection was evaluated using manually crafted injections embedded in web content or tool outputs, though the paper does not specify the injected or target prompts. GPT-4o-mini maintains utility, with absolute utility scores of 0.71 and 0.73 on the two benchmarks, respectively. However, even the existing Combined Attack achieves ASVs of 0.68 and 0.75. **These findings contradict prior claims and indicate that Instruction Hierarchy is ineffective when evaluated on diverse injected prompts.**

5.4 PromptGuard and Attention Tracker

Claims about the success of detection-based defenses should not rely solely on AUC: We evaluate two open-source detection-based defenses, PromptGuard [22] and Attention Tracker [15], measuring their FPRs, FNRs, and AUCs on the OpenPromptInjection and MMLU-PI benchmarks using the Combined Attack. As shown in Table 3(a) in the Appendix, both achieve high AUCs but exhibit significant limitations in terms of FPR or FNR. PromptGuard attains an AUC of 0.92 but an FPR of 0.89 on OpenPromptInjection, indicating a strong bias toward labeling most inputs as contaminated—detecting some attacks but producing excessive false alarms that undermine utility. Attention Tracker reaches AUCs of 1.00 on both benchmarks yet suffers a high FNR of 0.69 on the more challenging MMLU-PI benchmark. **These results show that AUC alone is insufficient for assessing the success of detection methods.**

Adaptive attacks: We propose a general adaptive attack framework for detection-based defenses. Given a target/injected prompt tuple (p_t, r_t, p_e, r_e), the framework optimizes a separator z to jointly (1) evade the detector D and (2) induce the LLM f to complete the injected task by generating r_e. The first objective enforces misclassification of contaminated data as clean, i.e., $D(x_t \| z \| p_e) = 0$, where x_t denotes the target data in p_t. We define an evasion loss $\ell_e(0, D(x_t \| z \| p_e))$, where a smaller value implies a higher probability of successful evasion. This loss can be customized depending on the targeted detector. The second objective ensures attack success through the standard cross-entropy loss introduced in Section 2, i.e., $\ell_{ce}(r_e, f(p_t \| z \| p_e)) = -\sum_{i=1}^{|r_e|} \log p_f(r_e^i \mid p_t \| z \| p_e \| r_e^{<i}))$. The total loss for adaptive attacks against detection defenses is defined as:

$$\mathcal{L}(p_t, p_e, r_e) = -\ell_e(0, D(x_t \| z \| p_e)) + \alpha \cdot \ell_{ce}(r_e, f(p_t \| z \| p_e)), \quad (3)$$

where α balances the two loss terms. Beyond optimizing the separator z, the framework can also optimize different components of the contaminated data, such as $z \| s_e$ (separator plus injected instruction) or $z \| p_e$ (separator, injected instruction, and injected data).

We apply our adaptive attack to Attention Tracker, which demonstrates strong detection performance in Table 3(a) in the Appendix. Attention Tracker uses a *focus score* to determine whether a sample is contaminated; thus, we define the evasion loss as this focus score and minimize it to reduce the likelihood of detection. To optimize the loss in Equation 3, we adopt GCG to iteratively update the tokens in z, $z \| s_e$, or $z \| p_e$. Following Attention Tracker's setup, we use Qwen2-1.5B-Instruct [41] as the LLM, set $\alpha = 0.01$, and evaluate the adaptive attacks using the same tuples (p_t, r_t, p_e, r_e) as in Section 5.2.

As shown in Table 3(b) in the Appendix, our adaptive attack substantially increases the FNRs of Attention Tracker, reaching 0.66 and 1.00 on the two benchmarks when optimizing only the separator z. All successfully evading samples also trigger the target LLM to produce the attacker-desired response r_e. When optimizing both the separator and injected instruction, the attack becomes even stronger. On OpenPromptInjection, FNR improves by 0.3. This improvement arises because modified instruction tokens make contaminated data appear less like explicit instructions, causing misclassification as clean. **Our results show that the perceived effectiveness of a detection-based defense can vary significantly between existing and adaptive attacks, highlighting the necessity of incorporating adaptive attacks in its evaluation.**

6 Conclusion

Recent defenses against prompt injection attacks were not evaluated in a principled way, which often overestimates their effectiveness and/or utility maintenance. In this paper, we highlight the need to comprehensively evaluate defenses along the two dimensions: effectiveness and general-purpose utility. We hope our study enables more rigorous evaluation for future prompt injection defenses.

Acknowledgements

We thank the anonymous reviewers for their valuable feedback and constructive suggestions. This work was supported in part by NSF Grants No. 2530786, 2450935, 2414406, and 2112562.

References

[1] Gabriel Alon and Michael Kamfonas. 2023. Detecting Language Model Attacks with Perplexity. *arXiv* (2023).

[2] Anish Athalye, Nicholas Carlini, and David Wagner. 2018. Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In *International conference on machine learning*. PMLR, 274–283.

[3] Md Ahsan Ayub and Subhabrata Majumdar. 2024. Embedding-based classifiers can detect prompt injection attacks. *arXiv preprint arXiv:2410.22284* (2024).

[4] Hezekiah J. Branch, Jonathan Rodriguez Cefalu, Jeremy McHugh, Leyla Hujer, Aditya Bahl, Daniel del Castillo Iglesias, Ron Heichman, and Ramesh Darwishi. 2022. Evaluating the Susceptibility of Pre-Trained Language Models via Handcrafted Adversarial Examples. *arXiv* (2022).

[5] Nicholas Carlini and David Wagner. 2017. Adversarial examples are not easily detected: Bypassing ten detection methods. In *Proceedings of the 10th ACM workshop on artificial intelligence and security*. 3–14.

[6] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2025. StruQ: Defending Against Prompt Injection with Structured Queries. In *USENIX Security Symposium*.

[7] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, and Chuan Guo. 2024. Aligning LLMs to Be Robust Against Prompt Injection. *arXiv preprint arXiv:2410.05451* (2024).

[8] Yann Dubois, Chen Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy S Liang, and Tatsunori B Hashimoto. 2024. Alpacafarm: A simulation framework for methods that learn from human feedback. *Advances in Neural Information Processing Systems* 36 (2024).

[9] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *AISec*.

[10] Rich Harang. 2023. Securing LLM Systems Against Prompt Injection. https://developer.nvidia.com/blog/securing-llm-systems-against-prompt-injection.

[11] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2020. Deberta: Decoding-enhanced bert with disentangled attention. *arXiv preprint arXiv:2006.03654* (2020).

[12] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* (2020).

[13] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. *Proceedings of the International Conference on Learning Representations (ICLR)* (2021).

[14] Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. 2024. PLeak: Prompt Leaking Attacks against Large Language Model Applications. In *CCS*.

[15] Kuo-Han Hung, Ching-Yun Ko, Ambrish Rawat, I-Hsin Chung, Winston H. Hsu, and Pin-Yu Chen. 2024. Attention Tracker: Detecting Prompt Injection Attacks in LLMs. *arXiv preprint arXiv:2411.00348* (2024).

[16] Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. 2023. Baseline Defenses for Adversarial Attacks Against Aligned Language Models. *arXiv* (2023).

[17] Xuechen Li, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. AlpacaEval: An Automatic Evaluator of Instruction-following Models. https://github.com/tatsu-lab/alpaca_eval.

[18] Xiaogeng Liu, Zhiyuan Yu, Yizhe Zhang, Ning Zhang, and Chaowei Xiao. 2024. Automatic and universal prompt injection attacks against large language models. *arXiv preprint arXiv:2403.04957* (2024).

[19] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and benchmarking prompt injection attacks and defenses. In *USENIX Security Symposium*.

[20] Yupei Liu, Yuqi Jia, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong. 2025. DataSentinel: A Game-Theoretic Detection of Prompt Injection Attacks. In *IEEE Symposium on Security and Privacy*.

[21] Alexandra Mendes. 2023. Ultimate ChatGPT prompt engineering guide for general users and developers. https://www.imaginarycloud.com/blog/chatgpt-prompt-engineering.

[22] Meta. 2024. PromptGuard Prompt Injection Guardrail. https://www.llama.com/docs/model-cards-and-prompt-formats/prompt-guard/.

[23] Yohei Nakajima. 2022. Yohei's blog post. https://twitter.com/yoheinakajima/status/1582844144640471040.

[24] OpenAI. 2022. OpenAI GPT-3.5 API [text-davinci-003]. https://platform.openai.com/docs/guides/text-generation.

[25] OpenAI. 2024. GPT-4o mini: advancing cost-efficient intelligence. https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/.

[26] OWASP. 2023. OWASP Top 10 for Large Language Model Applications. https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-2023-v1_1.pdf.

[27] Dario Pasquini, Martin Strohmeier, and Carmela Troncoso. 2024. Neural exec: Learning (and learning from) execution triggers for prompt injection attacks. In *Proceedings of the 2024 Workshop on Artificial Intelligence and Security*. 89–100.

[28] Fábio Perez and Ian Ribeiro. 2022. Ignore Previous Prompt: Attack Techniques For Language Models. In *NeurIPS ML Safety Workshop*.

[29] Fábio Perez and Ian Ribeiro. 2022. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527* (2022).

[30] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS*.

[31] Sander Schulhoff. 2023. Instruction defense. https://learnprompting.org/docs/prompt_hacking/defensive_measures/instruction.

[32] Sander Schulhoff. 2023. Sandwitch defense. https://learnprompting.org/docs/prompt_hacking/defensive_measures/sandwich_defense.

[33] Zedian Shao, Hongbin Liu, Jaden Mu, and Neil Zhenqiang Gong. 2024. Making LLMs Vulnerable to Prompt Injection via Poisoning Alignment. *arXiv preprint arXiv:2410.14827* (2024).

[34] Jiawen Shi, Zenghui Yuan, Yinuo Liu, Yue Huang, Pan Zhou, Lichao Sun, and Neil Zhenqiang Gong. 2024. Optimization-based prompt injection attack to llm-as-a-judge. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 660–674.

[35] Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. 2025. Prompt Injection Attack to Tool Selection in LLM Agents. *arXiv preprint arXiv:2504.19793* (2025).

[36] R Gorman Stuart Armstrong. 2023. Using GPT-Eliezer against ChatGPT Jailbreaking. https://www.alignmentforum.org/posts/pNcFYZnPdXy L2RfgA/using-gpt-eliezer-against-chatgpt-jailbreaking.

[37] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. The instruction hierarchy: Training llms to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208* (2024).

[38] Simon Willison. 2022. Prompt injection attacks against GPT-3. https://simonwillison.net/2022/Sep/12/prompt-injection/.

[39] Simon Willison. 2023. Delimiters won't save you from prompt injection. https://simonwillison.net/2023/May/11/delimiters-wont-save-you.

[40] Tong Wu, Shujian Zhang, Kaiqiang Song, Silei Xu, Sanqiang Zhao, Ravi Agrawal, Sathish Reddy Indurthi, Chong Xiang, Prateek Mittal, and Wenxuan Zhou. 2024. Instructional Segment Embedding: Improving LLM Safety with Instruction Hierarchy. *arXiv preprint arXiv:2410.09102* (2024).

[41] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2. 5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).

[42] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. 2023. Benchmarking and defending against indirect prompt injection attacks on large language models. *arXiv preprint arXiv:2312.14197* (2023).

[43] Yiming Zhang and Daphne Ippolito. 2023. Prompts should not be seen as secrets: Systematically measuring prompt extraction attack success. *arXiv preprint arXiv:2307.06865* (2023).

[44] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043* (2023).

Appendix

Table 3: Results of detection-based defenses on OpenPromptInjection and MMLU-PI.

(a) FPRs, FNRs, and AUCs of PromptGuard and Attention Tracker

Detection-based defense	OpenPromptInjection			MMLU-PI		
	FPR	FNR	AUC	FPR	FNR	AUC
PromptGuard	0.89	0.00	0.92	0.84	0.00	0.75
Attention Tracker	0.00	0.00	1.00	0.00	0.69	1.00

(b) FNRs of different adaptive attack strategies against Attention Tracker

Adaptive attack strategy	OpenPromptInjection	MMLU-PI
Separator	0.66	1.00
Separator+Instruction	0.96	1.00
Separator+Instruction+Data	0.96	1.00